

Evaluating AI-Generated Text Detection:

Analysis of DetectGPT, DIPPER and Retrieval-Based Defense

Davide Pietragalla, 1965270, *Sapienza University of Rome*,
Kevin Huang, 1993737, *Sapienza University of Rome*,

Abstract—The increasing fluency and widespread usage of large language models (LLMs) have been accompanied by a rise in malicious applications, such as fake content creation and academic plagiarism. This trend highlights the need for tools that aid in the detection of LLM-generated text. We will evaluate state-of-the-art approaches to AI-generated text detection, such as DetectGPT, and its robustness when subjected to stress from a parameter paraphrase generation model like DIPPER. Our objective is to assess the robustness of the AI-generated text detection, taking into account retrieval-based defenses. Finally, we will provide some suggestions for future developments.

Index Terms—Machine Learning, plagiarism, detection, NLP, DetectGPT, DIPPER, Retrieval-Based Defense



1 INTRODUCTION

The rapid advancement of Large Language Models (LLMs) have radically changed the NLP field, however it has simultaneously catalyzed an escalation in malicious applications. Higher-quality AI-generated text is becoming increasingly indistinguishable from human writing.

To address this challenge, diverse methods have been developed, which are divided in classifier-based model, watermarking techniques, and statistics-based approaches. Among these, DetectGPT exploit the structural properties of an LLM’s probability function, in which AI-generated texts tend to occupy areas of negative curvature. Despite it, modern discourse-level paraphrasing attacks is still being a threat, such as DIPPER, an advanced paraphraser that is able to preserve the text’s underlying semantics. In response to it, a retrieval-based defense can be a highly counter-strategy; it operates on a dynamically updated database with previously generated sequences maintained by LLM providers via API.

All these topics will be covered in this report, we analyzed and reported the following papers: “DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature” [22] and “Paraphrasing evades detectors of AI-generated text, but retrieval is an effective defense” [14].

2 TAXONOMY & RELATED WORK

In this section, we provide a comprehensive overview of existing approaches for detecting AI-generated text.

2.1 Classifier-Based Methods

As the reader may know, LLM-generated text detection aims to distinguish whether a piece of text was produced by an LLM, which is essentially a binary classification task.

2.1.1 Features-Based Classifiers

Features-based classifiers can focus on language features or model features.

For what concerns language features-based classifiers, the process typically starts with the extraction of key statistical language features, followed by the application of machine learning techniques to train a classification model. These features can be extensively categorized into stylistic features, complexity features, semantic features, psychological features, and knowledge-based features, and they are primarily obtained through statistical methods.

An interesting example of this methodology is provided by Shah et al. [30], who developed a classifier leveraging an ensemble of machine learning models, including logistic regression, decision trees, random forests, support vector classifiers, and gradient boosting.

A further example is the one proposed by Mindner et al. [21], who employed a multidimensional framework to enhance the classifier’s discriminative power. Their methodology leverages a wide list of features, including complexity metrics, semantic analysis, error-based features, list searches, readability assessments, alongside artificial intelligence feedback and text vectorization.

Although classifiers based on linguistic features have demonstrated significant efficacy, their shortcomings cannot be overlooked. The results from Shah and Mindner indicate that such feature classifiers have poor robustness against ambiguous semantics and often underperform compared to neural networks. Moreover, although classifiers based on stylistic features are capable of distinguishing between texts written by humans and those generated by LLMs, their ability to detect misinformation generated by LLMs is actually limited. This limitation is also highlighted by Schuster et al. [29], which proves that language models tend to produce stylistically coherent texts.

The branch of model features-based classifiers is not only capable of detecting texts generated by LLMs but can also be employed for text provenance tracing.

One interesting approach, proposed by Wang et al. [34] involves utilizing log-probability lists extracted from white-

box LLMs as features for AI-generated text (AIGT) detection. The authors assert that these probability sequences are analogous to waves in speech processing, which standard LLMs are not capable of analyzing effectively. Therefore, they designed a context network that combines a CNN with a two-layer Transformer. This architecture encodes the text by processing these sequences and identifies AI-generated content through a sequence tagging mechanism.

Another approach is the one introduced by Wu et al. [38], which involves a multi-task learning framework designed to simultaneously address text detection and authorship attribution. This method leverages a combination of features such as log likelihood, log rank, entropy, and LLM bias to train a neural network classifier.

However, a significant drawback of these methods is that they all require access to the source model’s logits. Therefore, for powerful closed-source models where logits are inaccessible, these methods remain ineffective.

2.1.2 Pre-Training Classifiers

These methods involve fine-tuning Transformer-based LMs, applying contrastive learning, or employing adversarial learning.

One interesting approach involves fine-tuning the model on a specific domain. Although recent studies, such as Guo et al., Liu et al., Chen et al., and Wang et al., [5], [8], [17], [35] achieve average accuracies of approximately 95%, these models are prone to overfitting.

Another astute approach is primarily used to address data scarcity. Since it works by learning a representation space where similar AI-generated texts are clustered together, these models remain robust against unseen generative models.

Bhattacharjee et al. [2] proposed ConDA, a contrastive domain adaptation framework, which combines standard domain adaptation technology with the representation capabilities of contrastive learning, greatly improving the model’s defense capabilities against unknown models.

The final approach focuses on making detectors less vulnerable to attacks such as paraphrasing. Common techniques include sample enhancement, where AI-generated texts are paraphrased to train the detector to recognize camouflaged content, and two-player games, in which an attacking paraphraser and a detector learn iteratively from one another to enhance performance.

A notable framework based on two-player games is OUTFOX, proposed by Koike et al. [13]. In this framework, the attacker utilizes the detector’s predicted labels as In-Context Learning (ICL) exemplars to generate text that poses significant detection challenges. Conversely, the detector treats this adversarially generated content as ICL exemplars to enhance its ability to identify sophisticated attackers. This reciprocal exchange of outputs creates a feedback loop that substantially improves the robustness of detectors against LLM-generated text.

2.1.3 LLMs-Based Detectors

Several works have examined the feasibility of utilizing LLMs as detectors to discern text generated by either themselves or other LLMs.

Empirical evidence suggests that the reliability of LLMs in detecting synthetic text is inversely proportional to their complexity. While earlier models like Grover demonstrated effective self-detection, more complex models such as GPT-4 fail significantly in zero-shot environments. Interestingly, these models exhibit contrasting biases: ChatGPT tends toward leniency, misclassifying AI-generated text as human in 50% of cases, whereas GPT-4 is overzealous, erroneously labeling 95% of human-written content as AI-generated. [14]

The transition from ‘native’ detection to In-Context Learning (ICL) marks a critical methodological shift. Frameworks like OUTFOX demonstrate that the core issue is not a lack of comprehension, but rather a lack of reference. By providing exemplars within the prompt, LLMs can overcome intrinsic limitations and even outperform specialized RoBERTa-based detectors. Ultimately, combining ICL with two-player adversarial dynamics suggests that effective detection requires a ‘learning-to-learn’ system capable of evolving alongside increasingly sophisticated generative models. [38]

2.2 Statistics-Based Methods

Within the statistics-based framework, detection methods leverage the inherent probabilistic signatures of language models to identify machine-generated text without the need for supervised training. Unlike approaches that rely on manual feature engineering, these methods focus on the statistical regularities of token distributions, such as log-likelihood, rank, and entropy. By analyzing the “white-box” output probabilities of a model or the “black-box” curvature of the probability space, these techniques discern the subtle discrepancies between the predictive patterns of LLMs and the more irregular nature of human writing.

2.2.1 White-Box Statistics

White-box methods for detecting text generated by LLMs require direct access to the source model for implementation. The existing white-box detection techniques primarily use zero-shot approaches, which involve obtaining the model’s logits output and calculating specific metrics. These metrics are then compared against predetermined thresholds obtained through statistical methods to identify LLM-generated text.

Some white-box statistical approaches detect LLM-generated text by comparing the differences in performance metrics after statistical perturbation. Mitchell et al. [18] proposed a method to identify text produced by LLMs by analyzing structural patterns in the LLMs’ probability functions, specifically in regions of negative curvature. The premise is that LLM-generated text tends to cluster at local log-probability maxima. Detection involves comparing log-probabilities of text against those from the target LLM, using a pre-trained mask-filling model like T5 to create semantically similar text perturbations. While innovative and sometimes more effective than supervised methods, DetectGPT has limitations, including potential performance drops if rewrites don’t adequately represent the space of meaningful alternatives, and high computational demands, as it needs to score many text perturbations. [38]

Bao et al. [1] reported a method that replaces the perturbation step of DetectGPT with a more efficient sampling step, significantly improving the detection accuracy by about 75% and increasing the detection speed by 340 times.

In contrast to DetectGPT, the white-box configuration in DNA-GPT by Yang et al. [39] utilizes large language models such as ChatGPT to continue writing truncated texts instead of employing perturbation settings. It analyzes the differences between the original text and the continued text by calculating probability divergence, achieving a detection performance close to 100%.

DetectLLM by Su et al. [31] is another recent contribution, that parallels the conceptual framework of DetectGPT. It employs normalized perturbed log-rank for text detection generated by LLMs, asserting a lower susceptibility to the perturbation model and the number of perturbations compared to DetectGPT.

2.2.2 Black-Box Statistics

Unlike white-box statistical methods, black-box statistical methods utilize a black-box model to calculate specific feature scores of a text without needing access to the logits of the source or surrogate model.

One example of this approach is the one provided by Mao et al. [19], who identified LLM-generated text by computing the similarity scores between the original texts and their rewritten and revised versions. These approaches are based on the observation that human-written texts tend to trigger more revisions when LLMs are tasked with rewriting and editing than LLM-generated texts.

However, the approaches of black-box statistics are not without challenges, including the substantial overhead of accessing the LLM and long response times. [38]

2.3 Watermarking

A watermark is a modification to the generated text that can be detected post-hoc by an algorithm while remaining imperceptible to human readers. Effective watermarks are difficult to remove and have little effect on the quality of generated text. [15]

2.3.1 Data-Driven Watermarking

These methods typically rely on a sort of backdoor insertion, where a small number of watermarked samples are included in the dataset in order to make the model learn a secret function set by the defender. When a specific trigger is activated, the backdoor watermark is activated, which is usually implemented in a black-box setting. This solution protects the model from unauthorized fine-tuning or use beyond the terms of the license. Even if the model is fine-tuned for several downstream tasks, the watermark remains difficult to eradicate. [7]

It is important to note that data-driven methods were originally thought to protect copyright of datasets, and therefore, they usually lack substantial payload capacity and generalizability. Furthermore, applying these approaches in the field of LLM-generated text detection requires significant resource investment, such as the embedding of watermarks in a very large amount of data and the retraining of LLMs. [38]

2.3.2 Model-Driven Watermarking

Model-driven methods integrate watermarks by modulating the logits output distribution or token-sampling parts during the inference process. Consequently, the LLMs return answers that inherently carry these embedded watermarks, allowing for their subsequent identification. [38]

This specific branch is a sort of plug-and-play solution that offers a functional and reliable way to protect models without impacting text quality or adjusting parameters.

Two central approaches in this category are those proposed by Kirchenbauer et al. [12] and Liu et al. [16]. The former pioneered a logits-based framework that biases the sampling process toward a randomly selected set of ‘green’ tokens. While effective and minimally invasive to text quality, this method’s dependence on a symmetric secret key introduces vulnerabilities, as key exposure can lead to watermark removal or forgery. To enhance robustness and security, Liu et al. advanced the field with two significant improvements. First, they proposed a semantic invariant method that utilizes semantic embeddings of preceding tokens to determine watermark logic, ensuring the watermark remains detectable even after synonym substitution or paraphrasing. Second, they addressed the security flaws of traditional keys by developing a private watermarking algorithm. Such a system employs two different neural networks for generation and detection, sharing specific parameters to optimize efficiency while eliminating the need for a vulnerable secret key.

Another noteworthy case is SemStamp by Hou et al., which is a robust sentence-level semantic watermarking algorithm that utilizes Locality-Sensitive Hashing (LSH). Unlike previous methods, SemStamp operates by partitioning the semantic embedding space into watermarked and non-watermarked regions. During the generation, this system repeatedly tests candidate sentences until the chosen output falls within a designated “marked” region of the embedding space. This approach has proven to be more resilient against paraphrase attacks. [9]

2.3.3 Post-Processing Watermarking

This last alternative of watermarking refers to a technique that involves embedding a watermark by processing the text after it has been output by a LLM. This solution functions as a separate module that is included in a pipeline with the output of the generative model.

This approach may have significant and various applications depending on specific cases. For instance, the proprietary and closed-source architecture of models such as GPT-5 makes internal watermarking nearly impossible. Nevertheless, post-processing watermarking offers a viable alternative by allowing third parties to independently embed watermarks into the outputs of black-box LLMs. [38]

3 OVERVIEW OF DETECTGPT

Considering the taxonomy we provided, DetectGPT by Mitchell et al. [18] falls into the category of statistics-based methods, more precisely in the white-box branch, and, like almost all approaches in this branch, it is zero-shot. As a statistical approach, it does not require training a separate classifier, collecting a dataset of real or generated passages, or explicitly watermarking generated text.

3.1 The Idea

DetectGPT is based on a simple hypothesis: minor rewrites of model-generated text tend to have a lower log probability under the model than the original sample, while minor rewrites of human-written text may have higher or lower log probability than the original sample. In other words, unlike human-written text, model-generated text tends to lie in areas where the log probability function has negative curvature. The hypothesis was empirically verified by Mitchell et al., who found that it holds true across a diverse body of LLMs, even when the minor rewrites, or perturbations, come from alternative language models. [18]

3.2 How it Works

The entire framework can be summarized into an algorithm, which in turn exploits pre-trained mask-filling models in order to generate passages that are ‘nearby’ the candidate passage. However, these mask-filling models are used off-the-shelf, without any fine-tuning or adaptation to the target domain.

Algorithm 1 DetectGPT model-generated text detection

- 1: **Input:** passage x , source model p_θ , perturbation function q , number of perturbations k , decision threshold ϵ
 - 2: $\tilde{x}_i \sim q(\cdot | x)$, $i \in [1..k]$ //mask spans, sample replacements
 - 3: $\tilde{\mu} \leftarrow \frac{1}{k} \sum_i \log p_\theta(\tilde{x}_i)$ //approx. expectation
 - 4: $\hat{d}_x \leftarrow \log p_\theta(x) - \tilde{\mu}$ //estimate $d(x, p_\theta, q)$
 - 5: $\hat{\sigma}_x^2 \leftarrow \frac{1}{k-1} \sum_i (\log p_\theta(\tilde{x}_i) - \tilde{\mu})^2$ //variance for normalization
 - 6: **if** $\frac{\hat{d}_x}{\sqrt{\hat{\sigma}_x^2}} > \epsilon$ **then**
 - 7: **return** true //probably model sample
 - 8: **else**
 - 9: **return** false //probably not model sample
-

Starting with a candidate text passage, the system first generates several alternative versions, referred to as perturbations, by applying minor modifications via a mask-filling model. Subsequently, it queries the source model to evaluate the probabilities of both the original text and its variants. The core of the method lies in comparing these values: the algorithm measures the discrepancy between the original text’s score and the average score of the perturbations. If this difference is sufficiently pronounced and exceeds a specific decision threshold, it indicates that the original text was located in an anomalous probability “peak,” leading the algorithm to classify it as synthetic. [18]

3.3 Why it Works

As previously stated, DetectGPT is based on the hypothesis that samples generated by a source model p_θ typically reside in regions of negative curvature within its log-probability function, a characteristic not shared by human-written text. In other words, if we apply minor perturbations to a passage $x \sim p_\theta$ to produce a variant $\tilde{x}$, the difference $\log p_\theta(x) - \log p_\theta(\tilde{x})$ is expected to be substantially larger, on average, for machine-generated outputs than for human text.

To exploit this hypothesis, they first define a perturbation function $q(\cdot | x)$ that yields a distribution over $\tilde{x}$,

representing slightly modified versions of x that retain the original meaning (assuming x is roughly paragraph-length). For instance, $q(\cdot | x)$ could be conceptualized as asking a human to rewrite a single sentence of x while preserving its semantics. Using this function, we can formally define the perturbation discrepancy, $d(x, p_\theta, q)$, as follows:

$$d(x, p_\theta, q) \triangleq \log p_\theta(x) - \mathbb{E}_{\tilde{x} \sim q(\cdot | x)} [\log p_\theta(\tilde{x})]$$

Mitchell et al. demonstrate that the perturbation discrepancy is an approximation of the local curvature of the model’s log-probability function around the candidate passage. More specifically, it is proportional to the negative trace of the Hessian matrix of the log-probability function.

To bridge the gap between discrete text tokens and continuous mathematical derivatives, the authors conceptualize the candidate passages as residing within a continuous latent semantic space. In this space, small vector displacements correspond to valid textual changes that preserve the original meaning.

The mathematical equivalence can be explained through the following steps:

- **Trace Estimation:** Computing the exact Hessian matrix for large language models is computationally intractable. To overcome this, the authors rely on Hutchinson’s trace estimator, which approximates the trace of a matrix using the expected value of directional second derivatives evaluated alongside random noise vectors z .
- **Differences Approximation:** Since the true directional second derivative is unavailable, it is approximated using finite differences. This involves evaluating the log-probability function f at the original point x , as well as at small displacements forward $f(x + z)$ and backward $f(x - z)$ within the semantic space.
- **Symmetric Noise Assumption:** Assuming the perturbation distribution is symmetric (i.e., moving in any semantic direction is equally probable), the complex finite difference equation simplifies dramatically. Therefore, the approximation of the negative Hessian trace becomes directly proportional to:

$$f(x) - \mathbb{E}_z[f(x + z)].$$

There, $f(x)$ corresponds to the log-probability of the original text, and $\mathbb{E}_z[f(x + z)]$ corresponds to the expected log-probability of the perturbed passages. Consequently, by calculating the perturbation discrepancy, DetectGPT mathematically estimates the “sharpness” of the local probability peak along the manifold of meaningful text.

Finally, it is worth noting that perturbations cannot be generated by randomly selecting tokens from a vocabulary. Instead, sampling within the semantic space ensures that all generated samples stay near the data manifold, resulting in meaningful and natural variations. [18]

3.4 Datasets

They rely on six datasets that encompass a wide range of everyday domains and LLM use cases.

Specifically, they use news articles from the XSum dataset (Narayan et al., 2018) [23] to represent fake news

detection, Wikipedia paragraphs from SQuAD contexts (Rajpurkar et al., 2016) [27] to simulate machine-written academic essays, and prompted stories from the Reddit WritingPrompts dataset (Fan et al., 2018) [6] to represent the detection of machine-generated creative writing submissions.

To assess robustness against distribution shifts, they also incorporate the English and German splits of WMT16 (Bojar et al., 2016) [3], alongside long-form answers authored by human experts from the PubMedQA dataset (Jin et al., 2019) [10].

3.5 Hyperparameters

The key hyperparameters for DetectGPT focus on the percentage of masked words, the length of those masked spans, and the specific model used to fill them. Drawing inspiration from existing masked language models, they consistently set the mask rate to 15% and the span length to two words across all experiments without further tuning. To fill in these masked sections, they relied on various versions of the T5 model, adapting the specific size or multilingual capability based on the language of the experiment and available computing resources, while leaving the generation parameters at their standard, untuned settings. [18]

3.6 Evaluation & Results

The detection capability of the approach was benchmarked against zero-shot baselines and supervised detectors, evaluated primarily through the AUROC metric. Zero-shot baselines are outperformed by an average of +0.06 AUROC, while a higher degree of generalization is guaranteed compared to supervised models, whose performance heavily degrades across domain and language shifts.

	PMQA	XSum	WritingP	Avg.
RoB-base	0.64 / 0.58	0.92 / 0.74	0.92 / 0.81	0.77
RoB-large	0.71 / 0.64	0.92 / 0.88	0.91 / 0.88	0.82
$\log p(x)$	0.64 / 0.55	0.76 / 0.61	0.88 / 0.67	0.69
DetectGPT	0.84 / 0.77	0.84 / 0.84	0.87 / 0.84	0.83

Fig. 1: DetectGPT demonstrates competitive accuracy against state-of-the-art supervised detectors when evaluating massive 175B-parameter models like GPT-3.

Evaluations on paraphrase attacks modifying up to 24% of the generated text showed that the AUROC remains above 0.8; however, this result is highly dependent on how the paraphrasing was conducted. It is highly likely that in a real-world environment, the AUROC would plummet, not to mention that detection performance decreases significantly if the generative model is unknown.

Finally, it is interesting to note that the size of the perturbation model has a proportional impact on the accuracy of DetectGPT, and that the framework reaches its maximum effectiveness when generating at least 100 perturbations.

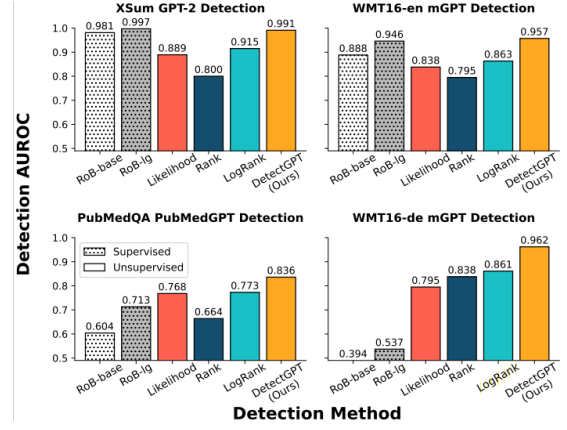


Fig. 2: Unlike supervised models that lose accuracy on out-of-domain topics or new languages, DetectGPT remains highly consistent.

4 OVERVIEW OF RETRIEVAL-BASED DEFENSE

In the following section, we provide the second main approach to AI-text detection examined in this report: Retrieval-Based Defense [14], which serves as an effective method against paraphrase-based attack such as DIPPER, that are meant to exploit the vulnerability of existing detectors.

4.1 DIPPER

DIPPER (Discourse Paraphraser) is a paraphrase-based attack designed to stress-test and evade existing detection mechanisms, such as the previous mentioned DetectGPT [18] and watermarking [7]. It was created by training an 11B parameter paraphrase generation model, since most existing AI detectors are vulnerable to semantic-preserving rewrites. For example, DetectGPT correctly detect 70.3% of AI-generated sequences from GPT2-XL, but it drops to only 4.6% after the text is paraphrased with DIPPER.

4.1.1 Advantages on modern paraphraser

Intuitively, paraphrasing alters the statistical properties of AI-generated text, which can fool outlier detection or classifiers while also reducing the number of watermarked tokens. DIPPER provides multiple advantages:

- **Long-Form Text Paraphrasing**, most modern paraphraser work only on sentence-level data, instead, DIPPER handles multiple sentences at the same time, leveraging discourse-level information, content re-ordering and context (such as the user prompt, which is involved in many critical LLM use cases).
- **Output Controllability**, an attacker may want to apply just the minimum amount of paraphrasing to evade a detector, but existing paraphraser lack an easy way to control output diversity. On the other hand, DIPPER provides two intuitive scalar control knobs at inference time.
- **Watermark Avoidance**, paraphrasing AI-generated text with a commercial LLM, the new version may still contain the provider’s watermark. To truly evade detection, an external and private model like DIPPER is required.

Method	XSum						SQuAD						WritingPrompts					
	GPT-2	OPT-2.7	Neo-2.7	GPT-J	NeoX	Avg.	GPT-2	OPT-2.7	Neo-2.7	GPT-J	NeoX	Avg.	GPT-2	OPT-2.7	Neo-2.7	GPT-J	NeoX	Avg.
$\log p(x)$	0.86	0.86	0.86	0.82	0.77	0.83	0.91	0.88	0.84	0.78	0.71	0.82	0.97	0.95	0.95	0.94	0.93*	0.95
Rank	0.79	0.76	0.77	0.75	0.73	0.76	0.83	0.82	0.80	0.79	0.74	0.80	0.87	0.83	0.82	0.83	0.81	0.83
LogRank	0.89*	0.88*	0.90*	0.86*	0.81*	0.87*	0.94*	0.92*	0.90*	0.83*	0.76*	0.87*	0.98*	0.96*	0.97*	0.96*	0.95	0.96*
Entropy	0.60	0.50	0.58	0.58	0.61	0.57	0.58	0.53	0.58	0.58	0.59	0.57	0.37	0.42	0.34	0.36	0.39	0.38
DetectGPT	0.99	0.97	0.99	0.97	0.95	0.97	0.99	0.97	0.97	0.90	0.79	0.92	0.99	0.99	0.99	0.97	0.93*	0.97
Diff	0.10	0.09	0.09	0.11	0.14	0.10	0.05	0.05	0.07	0.07	0.03	0.05	0.01	0.03	0.02	0.01	-0.02	0.01

TABLE 1: The table compares the AUROC scores of DetectGPT against four baseline zero-shot methods using 500 samples. DetectGPT consistently provides the most accurate detection across all models, from the 1.5B GPT-2 to the 20B GPT-NeoX.

4.1.2 How It Works

In order to train DIPPER, the authors contrusted a paraphrase dataset by leveraging the PAR3 dataset [32], which contains multiple translations of non-English novels into English aligned at a paragraph level. Given p and q aligned paragraphs:

$$\forall p \wedge q \mid p_1, p_2, \dots, p_N \in p \wedge q_1, q_2, \dots, q_M \in q \quad M \neq N$$

we can perform the following steps:

- 1) For each sentences of p find the corresponding alignment in q by using semantic similarity score from the paraphrase similarity metric designed by Wieting et al. [36] (which has alternative reward function for optimizing NMT systems) to run the sequence alignment algorithm designed by Needleman and Wunsch [24], which uses dynamic programming.
- 2) Choose a subset of sentences $p_i \dots p_j$ and match it with the associated alignment $q'_i \dots q'_j$ in q .
- 3) Compute the diversity control codes between $p_i \dots p_j$ and the randomly re-ordered $shuffle(q'_i \dots q'_j)$: lexical diversity (L) using their unigram token overlap, and the order diversity (O) using the correlation of tokens of overlapping words, both normalized to values $\{0, 20, 40, 60, 80, 100\}$ representing their percentage.
- 4) Map $shuffle(q'_i \dots q'_j)$ to $p_i \dots p_j$, leveraging context from the rest of p and control codes using string concatenation. Let input = $shuffle(q'_i \dots q'_j)$:

$$\begin{aligned} \text{lexical} &= L, \text{order} = O \oplus p_1 \dots p_{i-1} \oplus \\ &\oplus \langle p \rangle \text{input} \langle /p \rangle \oplus p_{j+1} \dots p_N \rightarrow p_i \dots p_j \end{aligned}$$

Where $\oplus$ is the string concatenation operation. In addition, during inference we can paraphrase any sequence of sentences, by marking it with $\langle p \rangle$ tags and assigning the desired control codes.

This process constructs DIPPER, a sequence-to-sequence Transformer neural network [33] with 6.3M paraphrase pairs, initialized with the pretrained 11B parameter T5-XXL 1.1 checkpoint [26] and fine-tuned on the paraphrase generation data.

4.1.3 DIPPER Attack Experiments

The experiments of attacking detection algorithms with DIPPER are conducted using two evaluation metrics:

- Detection accuracy, namely how often the input text is correctly detected as AI-generated. This is measured by the AUC-ROC metric, since it's independent from the chosen detection threshold, with FPR fixed to 1%.

- Semantic similarity (Sim), whether the original and the paraphrased generations share approximately the same semantics. The author used the state-of-the-art semantic similarity P-SP [37], an embedding averaging model strongly robust against topically similar non-paraphrases. The average P-SP score on human paraphrase pairs in PAR3 is 0.76, which is the threshold considered for preserved semantics.

The attacks are conducted by sampling 300 tokens long generations from three language models before passing them through DIPPER: GPT2-XL [25], OPT-13B [40], and the text-davinci-003 (variant from GPT-3.5) [4]. These LMs are evaluated on two type of tasks:

- 1) Open-ended generation, where an LM generates a 300 token continuation to a two sentence prompt, sampled from the WikiText-103 database [20].
- 2) Long-form question answering, in which an LM answers a question with 300 word response. 500 questions from six popular domains were scraped from r/explainlikeimfive subreddit, and each is paired with its longest human written answer forming a 3K long-form Q&A pairs dataset.

The attack are done against five detectors with their default hyperparameters and respecting their mininum length specification: Watermarking, DetectGPT, GPTZero, OpenAI's text classifier and RankGenXL-all.

The input to DIPPER has the following form:

$$\text{lexical} = L, \text{order} = O \text{ prompt } \langle p \rangle \text{ generated-text } \langle /p \rangle$$

The main attack experiments is conducted using $L = 20, 40, 60$ and $O = 0, 60$. The AI-generated sequence, paraphrased sequence, and human-writted sequence are all truncated to match the shortest length among them, and, to ensure higher semantic preservation, long sequences are paraphrased three senteces at a time, keeping the previous paraphrased text in the generation context.

Across all diversity control codes, DIPPER significantly lowers detection accuracy while preserving input semantics with a 88%-99% P-SP score. For instance, paraphrasing GPT2-XL reduces watermarks detection accuracy from 100% to 57.2%, DetectGPT from 70.3% to a mere 4.6% and OpenAI's text classifier from 30.0% to 15.6%. A simple way to make the attack even more stronger is to sample multiple times from DIPPER, but it can only be done if an attacker has access to a detector, which may be a strong assumption.

4.2 Retrieval

Instead of analyzing the statistical properties of texts (as in DetectGPT) or hiding watermarkers, we can retrieve

over previously generated sequences from API interfaces provided by the companies that host and distribute the LLMs. Since paraphrasing approximately preserves input semantics, the interface searches over the entire database previously generated text using a semantic similarity scorer like P-SP [36] or a retriever like BM25 [28].

4.2.1 How It Works

Let f_{LM} be an LLM API that takes a prompt x as input and return a continuation o ; f_{ret} a retriever that embeds variable-length sequences into fixed-size vectors that represent the input semantics:

- 1) Building the database, let $x_1, \dots, x_N$ (N up to 15M) be the set of prompts previously fed to the API, and $o_i = f_{LM}(x_i)$ be the corresponding LLM output. The database $Y = [y_1, \dots, y_N]$ is constructed by encoding every LLM API output as $y_i = f_{ret}(o_i)$, it's dynamically updated and accessible to clients only via API query.
- 2) Querying the database, let $y' = f_{ret}(o')$ be the encoded vector of a candidate text. A client can query the database to determine if it was generated by the API f_{LM} . The API provider can choose a threshold T and check if the maximum similarity score of o' , of the entries in the database, exceeds it:

$$\text{output} = \text{score} > T, \quad \text{where} \quad \text{score} = \max_{i \in \{1, \dots, N\}} \frac{y' \cdot y_i}{\|y'\| \|y_i\|}$$

The inner product with a exact machine-generated text should always get a score of 1.0, while lowering T increases the detection rate of paraphrased text, but it increases also the false positive rate.

4.2.2 Retrieval Comparisons

The retrieval method was compared against the previously mentioned detection algorithms (Watermark [7], DetectGPT [18] and OpenAI's text classifier) on long-form Q&A, using three kinds of databases:

- 1) 3K sequences generated by a specific LM.
- 2) 9K sequences formed by concatenating the generations from GPT2-XL [25], OPT-13B [40], and the text-davinci-003 [4].
- 3) 46K sequences constructed by combining the 3K from (1) with 43K LLM responses from ShareGPT-Vicuna (an open-source collection of real conversations between human users and ChatGPT).

Across all LMs, retrieval results to be the most effective detector, obtaining a 100% detection accuracy on raw AI-generated text. It achieves an average accuracy of 97.1% on the highest-diversity paraphrases (L60, O60), against the next best alternative, watermarking, with an average of 48.4%; 95.2% for database(2); and 96.6% for (3).

An alternative attack strategy that might be adopted in practice is text shortening or text mixing [11], where attackers use only a substring of an LLM generated output and mix it with text from other sources or human written text. Retrieval-based detection achieved an average accuracy of 90.5% on raw random substrings, but drops to 62.5% on substrings that include DIPPER paraphrasing.

5 DISCUSSION & FUTURE DEVELOPMENTS

As can be deduced from the underlying mechanics of DetectGPT, paraphrasing attacks, the use of black-box LLMs, and the vast selection of these models—which makes it difficult to trace the provenance of AI-generated text—effectively render the approach obsolete.

We believe that in such scenarios, where every variable is stacked against the so-called "defender", every detail in the implementation of AI-generated text detection systems becomes crucial. Therefore, starting precisely from the aforementioned weaknesses, we can highlight some of the practical challenges that must be addressed, acknowledging the severe and widespread issues caused by the improper generation of text.

First and foremost, it would be necessary to establish a global standard for the detection of AI-generated text; this would drive research forward in a unified, regulated, and easily updatable effort.

Regulations mandating watermarking within the models themselves for the detection of artificially generated text, such as Article 50 of the EU AI Act, are not the only viable approach. Greater transparency must be ensured in the intrinsic operation of LLMs, advancing models that provide APIs where token log probabilities are available, thereby empowering statistical detection methods and gradually moving toward the phasing out of totally black-box models in recognition of the severe potential consequences of unregulated AI text generation.

From a practical standpoint, it would be highly beneficial to enhance DetectGPT with various perturbation models of a different nature, such as T5 and DIPPER, in order to broaden its effective range. Furthermore, integrating an ensemble approach based on different versions of DetectGPT with diverse surrogate models for black-box scenarios—potentially weighted according to their empirically evaluated scoring capabilities—may provide an additional layer of robustness.

Finally, remaining on a practical note, it is now evident that the aforementioned global standard—or any meaningful advancement—must stem from a comprehensive approach that leverages the most effective branches of the taxonomy outlined in Section 2. While this naturally entails prolonged detection times, we believe this is not necessarily a major drawback, provided the identification process can be transformed into a detailed and fully automated procedure.

Consider the example of a social network that wants to determine which captions were generated by an LLM. In this case, it would still be possible to leverage a hierarchy of automated processes—ranging from the least to the most computationally expensive—to determine in the short and long term, with increasing accuracy, whether or not the text was artificially generated.

We can also improve the retrieval's robustness, in fact the defense heavily relies on the premise that the candidate text is generated from an API provider's database. We should work on the retrieval system against open-source models where the generation history is untracked. As we saw, the system is still having hard time against structural manipulations like shortening and mixing attacks, these issues should

be addressed with a retrieval capable to isolate and flag these adversarial sequences without inflating the FPR.

6 EVALUATION OF FUTURE DEVELOPMENTS

We begin by explicitly defining the two developmental hypotheses outlined in the previous section:

- By using different versions of DetectGPT (let’s call it Multi-DetectGPT) with structurally diverse perturbation models and surrogate models in the case of a black-box scenario, we increase the probability of successfully identifying LLM-generated text when it is present.
- A tiered detection pipeline, structured across varying levels of analytical depth, can mitigate the computational overhead of advanced, multi-approach methods.

6.1 Experimental Setup for Hypothesis 1

To ensure consistency with previous literature, the datasets utilized would be the same as those described in Section 3. For each domain, a substantial corpus of human-written and AI-generated texts should be collected. Subsequently, each parallel version of DetectGPT must be independently calibrated; specifically, its decision threshold must be adapted to the statistical behavior of its respective perturbation model operating under various diversity control codes. Finally, the “Multi-DetectGPT” must be assessed following the same methodologies adopted for its predecessor.

6.2 Experimental Setup for Hypothesis 2

We simulate a real-world scenario of a social network aiming to filter textual content by identifying artificially generated text. For this purpose, we implement a four-tier evaluation pipeline:

- Tier 0 (Topic Identification): A preliminary semantic analysis is conducted to evaluate the underlying topic of the post. If the content is semantically related to subjects deemed sensitive or potentially harmful, the text is forwarded to Tier 1.
- Tier 1 (Real-Time Filtering): A lightweight and computationally efficient classifier is deployed to process all incoming texts instantaneously. This layer serves to identify obvious AI-generated content with minimal latency. If the model determines with high confidence that the text is artificial, the content is immediately obscured. Otherwise, it is forwarded to Tier 2 for deeper analysis.
- Tier 2 (Asynchronous Filtering): The “Multi-DetectGPT” ensemble is leveraged to identify further cases that can be filtered out. If the evaluated score exceeds the decision threshold (ϵ) of any single DetectGPT variant, the content is immediately obscured; otherwise, it is forwarded to Tier 3.
- Tier 3 (Human Review): Texts that yield borderline scores in Tier 2 are flagged for manual review in order to verify the veracity and legitimacy of the information.

Naturally, in this scenario, the experimental evaluation must determine the accuracy of the pipeline only up to Tier 2, since the human review is considered a supplementary layer.

7 CONCLUSIONS

In conclusion, the emergence of advanced paraphrasing methods like DIPPER, along with the extensive use of black-box language models, has significantly diminished the effectiveness of statistical detection approaches such as DetectGPT. Retrieval-Based Defense has proven to be a notably strong method, consistently delivering high accuracy even when assessing content that has been extensively paraphrased. To address current shortcomings, we provided some theoretical suggestions that may improve the robustness of current methodologies, and hypothesized eventual evaluation of these future developments.

CONTRIBUTION STATEMENT

Each member reviewed and refined all sections of this document written by the other members to ensure accuracy and reduce redundancy. Specifically, each member was responsible for the following parts:

- Davide Pietragalla: Taxonomy and Related Work, Overview of DetectGPT, Discussion & Future Developments, Evaluation, Conclusions.
- Kevin Huang: Introduction, Overview of DIPPER and Retrieval-Based Defense, Discussion & Future Developments.

AI USAGE STATEMENT & METHODOLOGY

Herein, we report our practical experience with LLMs and broader artificial intelligence tools, accompanied by pertinent clarifications concerning our drafting and writing methodology.

- Davide Pietragalla: LLMs were employed as an assistive tool to gather information, paraphrase sections, rectify grammatical errors and excessive redundancies, and retrieve direct links to relevant literature. I decided to merge the Background and Related Work sections, expanding them to include a comprehensive taxonomy. I chose this structure to better analyze the domain and provide a solid foundation for the report.
- Kevin Huang: LLMs were employed as an assistive tool to gather information, paraphrase sections, rectify grammatical errors and excessive redundancies, and retrieve direct links to relevant literature.

REFERENCES

- [1] G. Bao, Y. Zhao, Z. Teng, L. Yang, and Y. Zhang, “Fast-detectgpt: Efficient zero-shot detection of machine-generated text via conditional probability curvature,” *ArXiv*, vol. abs/2310.05130, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:263831345>
- [2] A. Bhattacharjee, T. Kumarage, R. Moraffah, and H. Liu, “Conda: Contrastive domain adaptation for ai-generated text detection,” *ArXiv*, vol. abs/2309.03992, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:261660497>

- [0] O. Bojar, R. Chatterjee, C. Federmann, Y. Graham, B. Haddow, M. Huck, A. Jimeno-Yepes, P. Koehn, V. Logacheva, C. Monz, M. Negri, A. N  v  l, M. Neves, M. Popel, M. Post, R. Rubino, C. Scarton, L. Specia, M. Turchi, K. M. Verspoor, and M. Zampieri, "Findings of the 2016 conference on machine translation," in *Conference on Machine Translation*, 2016. [Online]. Available: <https://api.semanticscholar.org/CorpusID:14421595>
- [4] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 1877–1901.
- [5] Y. Chen, H. Kang, V. J. Zhai, L. Li, R. Singh, and B. Ramakrishnan, "Gpt-sentinel: Distinguishing human and chatgpt generated content," *ArXiv*, vol. abs/2305.07969, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:258686680>
- [6] A. Fan, M. Lewis, and Y. Dauphin, "Hierarchical neural story generation," in *Annual Meeting of the Association for Computational Linguistics*, 2018. [Online]. Available: <https://api.semanticscholar.org/CorpusID:44134226>
- [7] C. Gu, C. Huang, X. Zheng, K.-W. Chang, and C.-J. Hsieh, "Watermarking pre-trained language models with backdooring," *ArXiv*, vol. abs/2210.07543, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:252907247>
- [8] B. Guo, X. Zhang, Z. Wang, M. Jiang, J. Nie, Y. Ding, J. Yue, and Y. Wu, "How close is chatgpt to human experts? comparison corpus, evaluation, and detection," *ArXiv*, vol. abs/2301.07597, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:255998637>
- [9] A. B. Hou, J. J. Zhang, T. He, Y. Wang, Y.-S. Chuang, H. Wang, L. Shen, B. V. Durme, D. Khashabi, and Y. Tsvetkov, "Semstamp: A semantic watermark with paraphrastic robustness for text generation," in *North American Chapter of the Association for Computational Linguistics*, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:263831179>
- [10] Q. Jin, B. Dhingra, Z. Liu, W. Cohen, and X. Lu, "PubMedQA: A dataset for biomedical research question answering," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 2567–2577. [Online]. Available: <https://aclanthology.org/D19-1259/>
- [11] J. Kirchenbauer, J. Geiping, Y. Wen, M. Shu, K. Saifullah, K. Kong, K. Fernando, A. Saha, M. Goldblum, and T. Goldstein, "On the reliability of watermarks for large language models," *arXiv preprint arXiv:2306.04634*, 2023.
- [12] —, "On the reliability of watermarks for large language models," *ArXiv*, vol. abs/2306.04634, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:259095643>
- [13] R. Koike, M. Kaneko, and N. Okazaki, "Outfox: Llm-generated essay detection through in-context learning with adversarially generated examples," in *AAAI Conference on Artificial Intelligence*, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:260091573>
- [14] K. Krishna, Y. Song, M. Karpinska, J. Wieting, and M. Iyyer, "Paraphrasing evades detectors of ai-generated text, but retrieval is an effective defense," *ArXiv*, vol. abs/2303.13408, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:257687440>
- [15] —, "Paraphrasing evades detectors of ai-generated text, but retrieval is an effective defense," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS ’23. Red Hook, NY, USA: Curran Associates Inc., 2023.
- [16] A. Liu, L. Pan, X. Hu, S. Li, L. Wen, I. King, and P. S. Yu, "A private watermark for large language models," *ArXiv*, vol. abs/2307.16230, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:271333036>
- [17] Y. Liu, Z. Zhang, W. Zhang, S. Yue, X. Zhao, X. Cheng, Y. Zhang, and H. Hu, "Argugpt: evaluating, understanding and identifying argumentative essays generated by gpt models," *ArXiv*, vol. abs/2304.07666, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:258179137>
- [18] E. M. Y. L. A. K. C. D. Manning and C. Finn, "Detectgpt: zero-shot machine-generated text detection using probability curvature," *ArXiv*, 2023.
- [19] C. Mao, C. Vondrick, H. Wang, and J. Yang, "Raidar: generative ai detection via rewriting," *ArXiv*, vol. abs/2401.12970, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:267095281>
- [20] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," in *International Conference on Learning Representations*, 2017. [Online]. Available: <https://arxiv.org/abs/1609.07843>
- [21] L. Mindner, T. Schlippe, and K. Schaaff, "Classification of human- and ai-generated texts: Investigating features for chatgpt," *ArXiv*, vol. abs/2308.05341, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:260775600>
- [22] E. Mitchell, Y. Lee, A. Khazatsky, C. D. Manning, and C. Finn, "Detectgpt: Zero-shot machine-generated text detection using probability curvature," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 202. PMLR, 2023, pp. 24950–24962. [Online]. Available: <https://ericmitchell.ai/detectgpt>
- [23] S. Narayan, S. B. Cohen, and M. Lapata, "Don't give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization," *ArXiv*, vol. abs/1808.08745, 2018. [Online]. Available: <https://api.semanticscholar.org/CorpusID:215768182>
- [24] S. B. Needleman and C. D. Wunsch, "A general method applicable to the search for similarities in the amino acid sequence of two proteins," *Journal of Molecular Biology*, vol. 48, no. 3, pp. 443–453, 1970.
- [25] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, 2019. [Online]. Available: <https://openai.com/blog/better-language-models/>
- [26] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020. [Online]. Available: <http://jmlr.org/papers/v21/20-074.html>
- [27] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, "Squad: 100,000+ questions for machine comprehension of text," in *Conference on Empirical Methods in Natural Language Processing*, 2016. [Online]. Available: <https://api.semanticscholar.org/CorpusID:11816014>
- [28] S. E. Robertson, S. Walker, S. Jones, M. M. Hancock-Beaulieu, and M. Gatford, "Okapi at trec-3," in *Proceedings of the Third Text REtrieval Conference (TREC-3)*, ser. NIST Special Publication, vol. 109, 1995, p. 109.
- [29] T. Schuster, R. Schuster, D. J. Shah, and R. Barzilay, "The limitations of stylometry for detecting machine-generated fake news," *Computational Linguistics*, vol. 46, no. 2, pp. 499–510, Jun. 2020. [Online]. Available: <https://aclanthology.org/2020.cl-2.8/>
- [30] A. Shah, P. Ranka, U. Dedhia, S. Prasad, S. Muni, and K. Bhowmick, "Detecting and unmasking ai-generated texts through explainable artificial intelligence using stylistic features," *International Journal of Advanced Computer Science and Applications*, vol. 14, no. 10, 2023. [Online]. Available: <http://dx.doi.org/10.14569/IJACSA.2023.01410110>
- [31] J. Su, T. Y. Zhuo, D. Wang, and P. Nakov, "Detectllm: Leveraging log rank information for zero-shot detection of machine-generated text," *ArXiv*, vol. abs/2306.05540, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:259129463>
- [32] K. Thai, M. Karpinska, K. Krishna, W. Ray, M. Inghilleri, J. Wieting, and M. Iyyer, "Exploring document-level literary machine translation with parallel paragraphs from world literature," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2022.
- [33] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez,  . Kaiser, and  . Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008. [Online]. Available: <http://papers.nips.cc/paper/7181-attention-is-all-you-need.pdf>
- [34] P. Wang, L. Li, K. Ren, B. Jiang, D. Zhang, and X. Qiu, "SeqXGPT: Sentence-level AI-generated text detection," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 1144–1156. [Online]. Available: [https://aclanth](https://aclanthology.org/2023.emnlp-main.73/)

- semantic similarity," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Florence, Italy: Association for Computational Linguistics, 2019, pp. 4344–4355. [Online]. Available: <https://aclanthology.org/P19-1427>
- [37] J. Wieting, K. Gimpel, G. Neubig, and T. Berg-Kirkpatrick, "Paraphrastic representations at scale," in *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Abu Dhabi, UAE: Association for Computational Linguistics, 2022, pp. 379–388. [Online]. Available: <https://aclanthology.org/2022.emnlp-demos.38>
- [38] J. Wu, S. Yang, R. Zhan, Y. Yuan, L. S. Chao, and D. F. Wong, "A survey on LLM-generated text detection: Necessity, methods, and future directions," *Computational Linguistics*, vol. 51, no. 1, pp. 275–338, Mar. 2025. [Online]. Available: <https://aclanthology.org/2025.cl-1.8/>
- [39] X. Yang, W. Cheng, L. Petzold, W. Y. Wang, and H. Chen, "Dna-gpt: Divergent n-gram analysis for training-free detection of gpt-generated text," *ArXiv*, vol. abs/2305.17359, 2023. [Online]. Available: <https://api.semanticscholar.org/CorpusID:258960101>
- [40] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin *et al.*, "Opt: Open pre-trained transformer language models," *arXiv preprint arXiv:2205.01068*, 2022. [Online]. Available: <https://arxiv.org/abs/2205.01068>